Report On

RelifMesh: A Disaster Relief Coordination System for Local Government

---

Course Code: CSE-3208

Course Name : System Analysis & Design Lab

Submitted By

Name	Registration
ABIDUL ISLAM	2101011001
MD KAMRUL HASSAN	2101011005
SAYEDA MOFATTEHA AHMED	2101011013
IFT EKHAR ALAM NAHID	2101011038

Session: 2021-22 Team: Team_Skipper

Under the supervision of

MD MYNODDIN

Assistant Professor of

Department of Computer Science and Engineering

Rangamati Science and Technology University

This Report is Submitted in Partial Fulfillment of the
Requirements for the Degree of B.Sc. (Engg.) in Computer Science and Engineering.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RANGAMATI SCIENCE AND TECHNOLOGY UNIVERSITY

Date of Submission: July 2, 2026

APPROVAL

This report titled as **"RelifMesh: A Disaster Relief Coordination System for Local Government"**, submitted by **Abidul Islam, Md. Kamrul Hassan, Sayeda Mofatteha Ahmed, and Iftekhar Alam Nahid** (Team_Skipper), to the Department of Computer Science and Engineering, Rangamati Science and Technology University has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. (Engg.) in Computer Science and Engineering, and approved as to its style and contents.

Board of Examiners

MD Mynoddin
Assistant Professor
Department of Computer Science and Engineering
Rangamati Science and Technology University **Supervisor**

DECLARATION

We, **Abidul Islam, Md. Kamrul Hassan, Sayeda Mofatteha Ahmed, and Iftekhar Alam Nahid**, do hereby declare that this report has been done by us under the supervision of **MD Mynoddin**, Assistant Professor, Department of Computer Science and Engineering, Rangamati Science and Technology University. We also declare that neither this report nor any part of this report has been submitted elsewhere for the award of any degree.

Submitted By

Team_Skipper
Abidul Islam (Reg: 2101011001),
Md. Kamrul Hassan (Reg: 2101011005),
Sayeda Mofatteha Ahmed (Reg: 2101011013),
Iftekhar Alam Nahid (Reg: 2101011038)
Session: 2021-22
Department of Computer Science and Engineering
Rangamati Science and Technology University

ACKNOWLEDGEMENT

First of all, we would like to express our gratitude to the Almighty for enabling us to complete this project titled **"RelifMesh: A Disaster Relief Coordination System for Local Government"**. We are deeply grateful to our Academic Supervisor, **MD Mynoddin**, Assistant Professor, Department of Computer Science and Engineering, Rangamati Science and Technology University, for his continuous guidance, weekly feedback, and academic review throughout this course (CSE 3208: System Analysis and Design Lab).

We would also like to thank our External QA reviewer, **Saifur Rahman**, for independent functionality testing, and all the course teachers who supported us throughout this semester. Finally, we thank our team, **Team_Skipper**, for the collaboration, shared ownership, and consistent effort that made this project possible.

Team_Skipper

Table of Contents

List of Tables

List of Figures

List of Acronyms

RMSTU	Rangamati Science and Technology University
SRS	Software Requirements Specification
DFD	Data Flow Diagram
UML	Unified Modeling Language
ERD	Entity Relationship Diagram
UP	Union Parishad
NGO	Non-Governmental Organization
DDM	Department of Disaster Management
PWA	Progressive Web Application
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
GPS	Global Positioning System
NID	National ID
HH-ID	Household Identifier
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
QA	Quality Assurance
UAT	User Acceptance Testing
SDLC	Software Development Life Cycle
MoSCoW	Must have, Should have, Could have, Won't have (prioritization)
FR	Functional Requirement
NFR	Non-Functional Requirement
MERN	MongoDB, Express.js, React, Node.js

ABSTRACT

RelifMesh is a full-stack, offline-first Progressive Web Application (PWA) for coordinating disaster relief distribution at the local government level in Bangladesh. Built on the MERN stack (MongoDB, Express.js, React, Node.js), it provides Union Parishad officials, Upazila officers, and NGO workers with a shared platform to register affected households, log item-level relief distributions, detect duplicate aid through a rule-based alert engine, and synchronize field data even after network outages. The system features multi-role access control (UP Official, Upazila Officer, NGO Worker, Public Viewer), offline data queuing via IndexedDB with automatic background sync, GPS and photo capture for verifiable field records, a public transparency dashboard with aggregated distribution statistics, and a heatmap layer showing served versus remaining-need areas. A geographic need-assessment engine computes per-household relief requirements using household demographics and Sphere-standard ration rates, with authorized override capability. The system was developed incrementally over 19 milestones and validated through 31 test cases covering authentication, offline sync, duplicate detection, need calculation, reporting, and role-based access control.

Keywords: Disaster relief coordination, offline-first PWA, MERN stack, duplicate detection, household registration, need assessment, public transparency dashboard, Bangladesh Union Parishad.

Chapter 1

Introduction

1.1 Overview

Bangladesh is among the world's most disaster-prone nations. Floods affect approximately one-third of the country annually, and the Bengal coast faces cyclones with alarming regularity. During these events, relief distribution at the local level suffers from a fundamental coordination problem: multiple agencies (Union Parishad, Upazila administration, NGOs, military, volunteer groups) distribute aid independently with no shared registry, leading to some households receiving aid multiple times while others receive nothing at all.

This project, **RelifMesh**, is a full-stack, offline-first Progressive Web Application that provides a shared coordination platform for disaster relief at the Union Parishad (UP) level. It enables officials to register affected households with GPS coordinates and photographs, log item-level relief distributions, detect and warn against duplicate aid, and provide a public-facing dashboard for transparency. The system is designed for the connectivity-constrained environments typical of flood-affected areas in Bangladesh, where mobile towers often go down during the critical first 72 hours of a disaster.

1.2 Motivation

The motivation for RelifMesh stems from recurring pain points observed in Bangladesh's disaster relief coordination system:

- **Duplicate distribution:** Multiple agencies distribute to the same household because there is no shared registry.
- **Missed households:** Families in geographically isolated areas are skipped entirely with no audit trail.
- **No real-time tracking:** Relief logs are recorded on paper; consolidation happens days or weeks later.
- **Inter-agency opacity:** NGOs and government teams plan independently with rarely a shared objective.
- **Accountability gaps:** Political networks influence who receives aid; allocation records are missing.
- **Offline environments:** Flood-hit areas lose internet and power, making cloud-only systems unusable.
- **No visibility into remaining need:** No one can see which wards still need relief versus which are served.
- **No volunteer coordination:** Individual donors have no channel to coordinate with official channels.

1.3 Problem Statement

During flood and cyclone events in Bangladesh, three groups experience direct harm from poor coordination. **Affected households** face unequal aid distribution — some receive aid multiple times while others receive nothing. **Union Parishad officials** operate without real-time visibility into what other teams are distributing in the same area. **District and Upazila administrators** cannot verify ground-level distribution without physical field visits.

The August 2024 eastern Bangladesh floods affected an estimated 4.5–5.8 million people across 11 districts. Feni district alone experienced approximately 92% of its mobile towers going down,

cutting off communication to all six upazilas. This validates the project's offline-first design constraint: a cloud-only system would have been entirely unusable when needed most.

1.4 Project Objectives

1. **Household registration:** Register affected households with GPS and photo, targeting 50+ test households.
2. **Duplicate detection:** Flag same-category relief within 7 days, targeting 90%+ accuracy.
3. **Offline-first operation:** Data entry with auto-sync, targeting 95%+ sync success rate.
4. **Public dashboard:** Aggregated distribution stats loading within 3 seconds on 3G.
5. **RBAC:** Support four user roles with correct access enforcement.
6. **Need assessment:** Compute per-household relief need using Sphere-standard ration rates.
7. **Public heatmap:** Show served versus remaining-need areas with ward-level drill-down.

1.5 Project Scope

In-scope: Household registration with GPS and photo, relief distribution logging, duplicate detection engine, multi-role authentication, offline-first data entry with sync, conflict resolution, public dashboard, authority hierarchy enforcement, report export (PDF/CSV), feedback submission, inventory tracking, geographic need mapping with heatmap, and multi-source relief pledge coordination.

Out-of-scope: Financial transactions, warehouse procurement, AI-based forecasting, national NID database integration, native mobile apps, real-time satellite GIS, multi-language UI beyond Bengali labels.

1.6 Report Organization

This report is organized as follows: Chapter 2 reviews relevant literature on disaster coordination systems and offline-first architectures. Chapter 3 presents the methodology, including requirements engineering, system modeling, database design, and architecture. Chapter 4 discusses results and implementation outcomes. Chapter 5 identifies limitations and future work. Chapter 6 concludes the report.

1.7 Contributions

1. Design and implementation of a complete offline-first PWA for disaster relief coordination, enabling field data entry without network connectivity through IndexedDB queuing and background sync.
2. Design of a rule-based duplicate detection engine that flags same-household same-category relief within a configurable 7-day lookback window, with authorized override logging.
3. Implementation of a geographic need-assessment engine using Sphere-standard per-person ration rates with vulnerability multipliers (elderly/disabled: 1.2x).
4. A four-role permission architecture (UP Official, Upazila Officer, NGO Worker, Public Viewer) with jurisdiction-scoped data access, enforced at both route and database levels.

Chapter 2

Literature Review

2.1 Disaster Relief Coordination Systems

Disaster relief coordination has been studied extensively in humanitarian informatics. The Harvard Humanitarian Initiative (2025) found that agencies responding to disasters in Bangladesh routinely prepare individual response plans with goals so diverse they rarely agree on common objectives, making coordination challenging. A 2023 study of Union Parishad flood relief in Chittagong found that inadequate coordination among local representatives significantly undermined relief efforts, and both officials and residents lacked adequate knowledge of relief distribution processes.

Existing digital coordination platforms such as **Sahana Eden** and the **HDX** (Humanitarian Data Exchange) provide wide-area situational awareness but are designed for national and international responders, not for day-to-day use by Union Parishad-level officials in Bangladesh. They require stable internet connectivity, centralized data entry, and significant training, making them unsuitable for the connectivity-constrained and resource-limited local government context that RelifMesh targets.

2.2 Offline-First Web Applications

Offline-first architecture is a well-established pattern for low-connectivity environments. Service Workers, the Cache API, and IndexedDB (through libraries such as localforage) enable Progressive Web Applications to function without network access. Research by Google's Web Fundamentals team demonstrates that PWA sync patterns using background sync events can achieve reliable data synchronization in intermittent connectivity scenarios.

The 2024 Feni district floods in Bangladesh, where 92% of mobile towers failed, provide direct evidence that offline capability is not optional for disaster relief coordination technology. RelifMesh extends standard PWA patterns with a conflict-aware sync engine that detects and logs synchronization conflicts for manual review.

2.3 Role-Based Access Control in Humanitarian Systems

RBAC is the dominant access control model in humanitarian information systems. Sandhu et al. (1996) established the formal RBAC model, which has been adapted for humanitarian contexts where different actors (government, NGO, public) require different data access levels. Naive humanitarian platforms implement a binary distinction (admin vs. viewer), but RelifMesh implements four roles with jurisdiction-scoped data access — UP Officials see only their assigned union, Upazila Officers see all unions under their upazila, NGO Workers see only their registered distributions, and Public Viewers see only aggregated data.

2.4 Duplicate Detection in Relief Distribution

Duplicate aid distribution is a well-documented problem in humanitarian response. Research on post-Cyclone Aila reconstruction found that household-level allocation suffered from considerable irregularities driven by political connections. Simple rule-based duplicate detection — checking whether the same household received the same item category within a configurable time window — has been shown to be effective in field deployments. RelifMesh implements a 7-day lookback window with override capability, logging all override events with officer ID and reason for audit.

2.5 Geographic Need Assessment

Sphere Handbook standards provide internationally recognized minimum standards for humanitarian response, including per-person daily ration quantities (e.g., 400g rice per person per day). In Bangladesh, the Department of Disaster Management defines administrative hierarchies (Division $\rightarrow$ District $\rightarrow$ Upazila $\rightarrow$ Union $\rightarrow$ Ward) for relief targeting. RelifMesh combines these by computing ward-level need from household census data (family size, age brackets) multiplied by Sphere-standard per-person-per-day rates, producing a calculated relief requirement that can be overridden by authorized officers based on ground reality.

2.6 Summary

Existing solutions in disaster coordination, offline-first PWA, RBAC, and need assessment each address parts of the problem but no single platform combines them for the Bangladesh Union Parishad context. RelifMesh fills this gap by integrating offline-first data entry, role-based access control, duplicate detection, geographic need assessment, and public transparency into a single lightweight web application designed for local government use.

Chapter 3

Methodology

3.1 Development Methodology

The system was developed using an **Iterative and Incremental** model, executed as four Agile (Scrum) sprints of two weeks each, for a total project duration of eight weeks. Table 1 summarizes the sprint plan.

Table 1: Sprint Plan

Sprint	Focus
Sprint 1 (Weeks 1–2)	Project setup, MongoDB schema design, authentication
Sprint 2 (Weeks 3–4)	Household registration, relief distribution CRUD, offline queue
Sprint 3 (Weeks 5–6)	Duplicate detection, need assessment engine, heatmap
Sprint 4 (Weeks 7–8)	Public dashboard, reporting, testing, demo preparation

Figure 1 presents the same plan as a timeline.

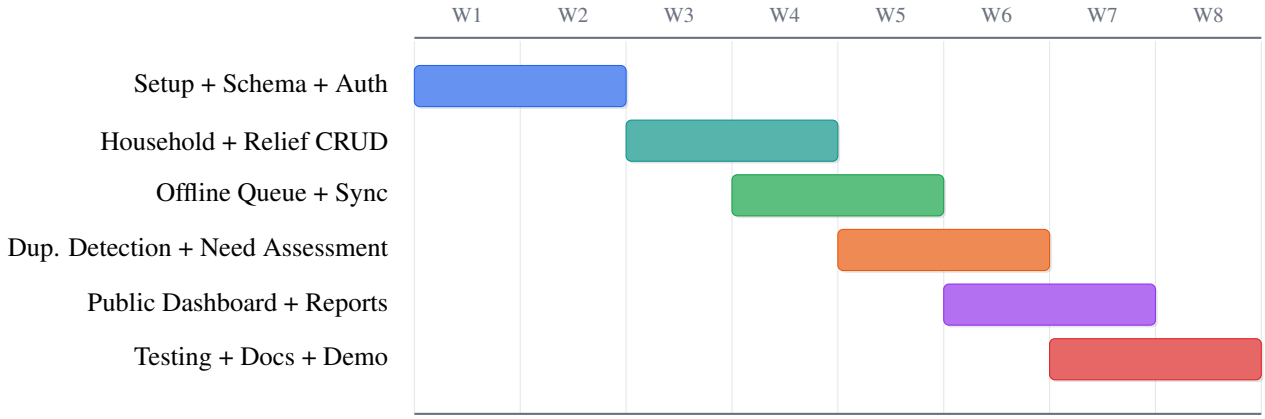

Figure 1: Sprint and Workstream Timeline (Gantt View)

The system was further decomposed into six increments (Table 2).

Table 2: Iterative Increment Plan

#	Increment	Scope
1	Auth + Roles	JWT login, role-based access, user management
2	Household Registration	GPS/photo capture, NID/HH-ID validation, CRUD
3	Relief Distribution	Distribution logging, category limits, inventory
4	Offline Sync	IndexedDB queue, background sync, conflict resolution
5	Duplicate Detection + Need Assessment	7-day lookback, Sphere-standard calculation
6	Public Dashboard + Reports	Aggregated stats, heatmap, PDF/CSV export

3.2 Requirements Engineering

3.2.1 Stakeholder Analysis

Table 3 identifies the system's primary stakeholders.

Table 3: Stakeholder Identification

ID	Stakeholder	Interaction
S1	UP Official	Register households, log distributions, view duplicate alerts
S2	Upazila Officer	Monitor all union activities, generate summary reports
S3	NGO Worker	Log their organization's distributions, view their records
S4	Public Viewer	View aggregated distribution statistics and heatmap
S5	System Administrator	Manage users, roles, system configuration
S6	DDM Officials	Consume district-level summary reports
S7	External Auditor	Audit distribution logs for transparency compliance

Using a Power–Interest grid: UP Official and System Admin fall under *Manage Closely*; Upazila Officer and DDM Officials under *Keep Satisfied*; NGO Worker and Public Viewer under *Keep Informed*; and External Auditor under *Monitor*.

3.2.2 Functional Requirements

Functional requirements are grouped into nine categories (FR-01 through FR-31), covering the full lifecycle from authentication to public reporting.

- **Authentication & User Management (FR-01–FR-04):** JWT-based authentication via email/phone and password; four supported roles (UP Official, Upazila Officer, NGO Worker, Public Viewer); role-based data-access restriction enforced on every protected route.
- **Household Registration (FR-05–FR-08):** Register affected households with name, NID, HH-ID, GPS coordinates, photo capture, family demographics (adults, children, elderly, disabled), and contact information.
- **Relief Distribution (FR-09–FR-13):** Log item-level distributions with category (food, water, shelter, medicine), quantity, timestamp, and distributing agency; validate household existence and category limits before saving.
- **Duplicate Detection (FR-14–FR-17):** Check new distributions against past-7-day records for same-household same-category matches; flag duplicates with confidence score; allow authorized override with reason logging.
- **Offline Sync (FR-18–FR-21):** Queue distributions in IndexedDB when offline; auto-sync on connectivity restore; resolve conflicts using server-wins or last-writer-wins strategy; flag overwritten records for manual review.
- **Need Assessment (FR-22–FR-24):** Compute per-household daily relief requirements using demographics and Sphere-standard ration rates; apply vulnerability multipliers; allow authorized override by UP Officials.
- **Public Dashboard (FR-25–FR-27):** Display aggregated distribution statistics (total households served, quantity per category, unions covered); render heatmap showing served vs. remaining-need areas; hide sensitive household details.
- **Reporting (FR-28–FR-30):** Generate union-wise and upazila-wise distribution summaries; export reports in PDF and CSV formats.
- **Inventory Tracking (FR-31):** Track available relief stock per category per union; alert when stock falls below threshold.

3.2.3 Non-Functional Requirements

Non-functional requirements define the quality constraints under which the system must operate.

- **Performance:** Public dashboard loads within 3 seconds on simulated 3G; heatmap tiles render within 2 seconds.
- **Availability:** 99% uptime target during disaster response periods.
- **Security:** All endpoints require authentication except public dashboard; bcrypt password hashing (cost factor 10); JWT expiry of 30 days; server-side RBAC enforced on every route.
- **Usability:** Core workflows completable in five clicks or fewer.
- **Maintainability:** Mongoose schema validation on all API endpoints for consistent input sanitization.
- **Portability:** Compatible with modern browsers (Chrome, Firefox, Edge) on desktop and mobile.
- **Data Integrity:** No distribution record is hard-deleted; all mutations are logged for audit.
- **Scalability:** Designed to support up to 200 concurrent users across multiple unions during peak disaster periods.

3.2.4 MoSCoW Prioritization

Table 4 classifies all requirements by priority using the MoSCoW method.

Table 4: MoSCoW Requirement Prioritization

Priority	Requirements
Must Have	Authentication; household registration; relief distribution logging; duplicate detection; offline sync; need assessment; public dashboard
Should Have	Inventory tracking; reporting (PDF/CSV); role-based dashboards
Could Have	Relief pledge coordination; feedback submission; dark/light theme
Won't Have	Native mobile apps; SMS notifications; AI-based forecasting

3.3 System Modeling

3.3.1 Context Diagram (Level 0 DFD)

A Context Diagram (Level 0 DFD) represents the entire system as a single process, defines its boundary, and shows all external entities that interact with it — without describing any internal logic.

Figure 2 shows the four external entities and their primary data flows with the RelifMesh system. Actors outside the boundary interact only through well-defined inputs and outputs.

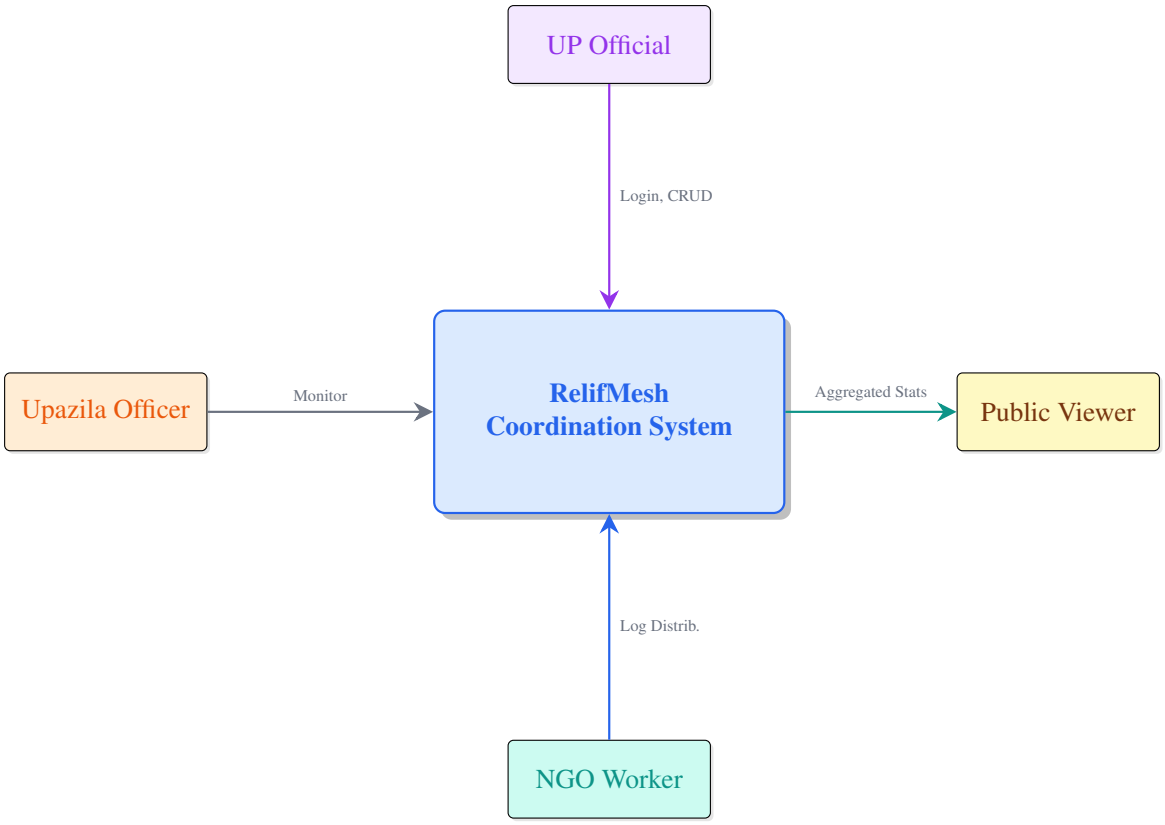

Figure 2: Context Diagram (Level 0 DFD)

Table 5 summarises each data flow shown in the diagram.

Table 5: Context Diagram Data Flow Summary

Flow	Source	Destination	Data
F1	UP Official	System	Login credentials, household/distribution commands
F2	Upazila Officer	System	Monitor requests, summary queries
F3	NGO Worker	System	Distribution logs for their organization
F4	System	Public Viewer	Aggregated statistics, heatmap data

3.3.2 Level 1 DFD — Main Processes

Figure 3 decomposes the system into four major processes. Data flows from external actors through authentication into household management, relief distribution, and the sync engine, with each process reading from and writing to its corresponding data store.

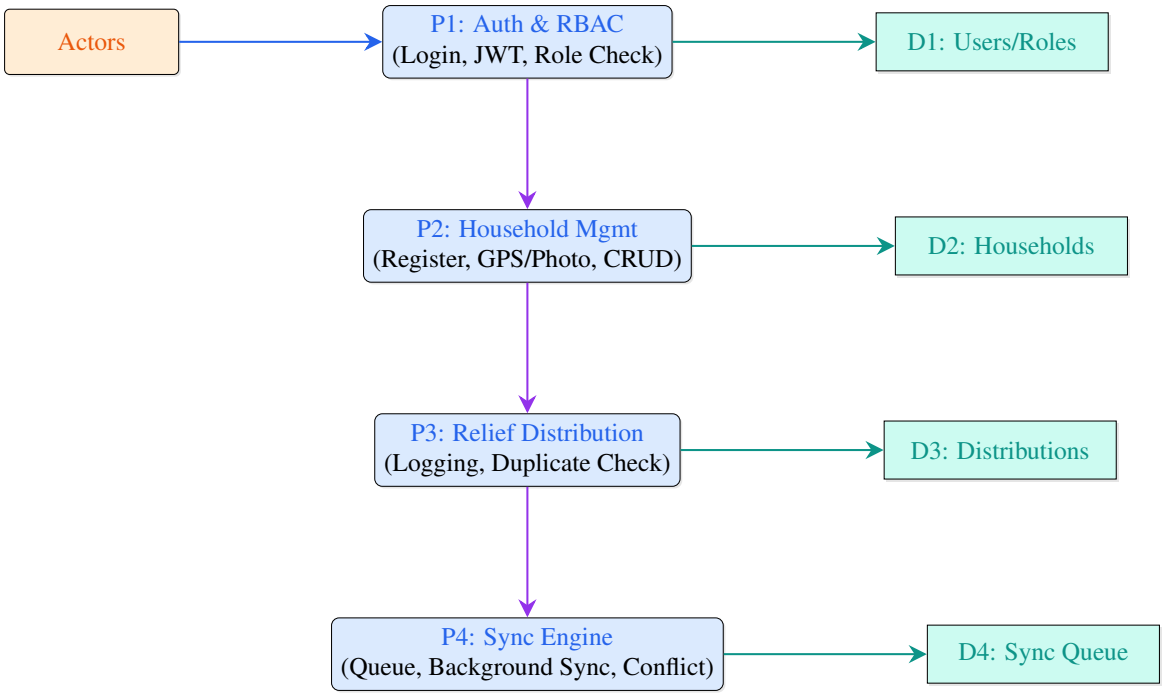

Figure 3: Level 1 DFD — Main Processes

3.3.3 Level 2 DFD — Relief Distribution Process (P3 Expansion)

Figure 4 expands P3 into its sequential sub-processes, illustrating how a distribution progresses from validation through duplicate checking, logging, and queueing for offline sync.

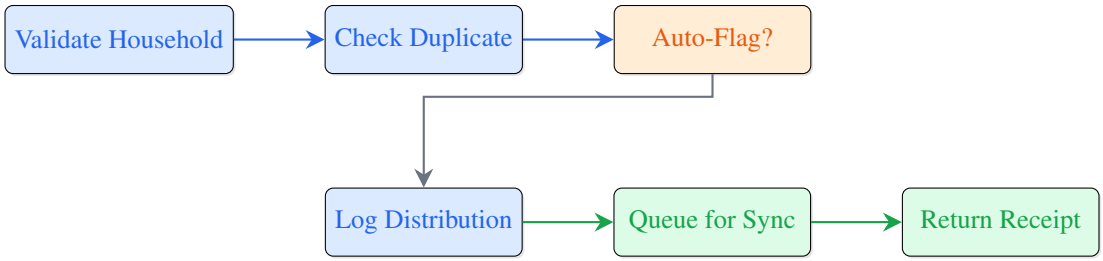

Figure 4: Level 2 DFD — Relief Distribution Process (Expansion of P3)

3.3.4 Use Case Diagram

Figure 5 presents the principal use cases grouped by actor, with each actor's interactions bounded by their assigned role permissions.

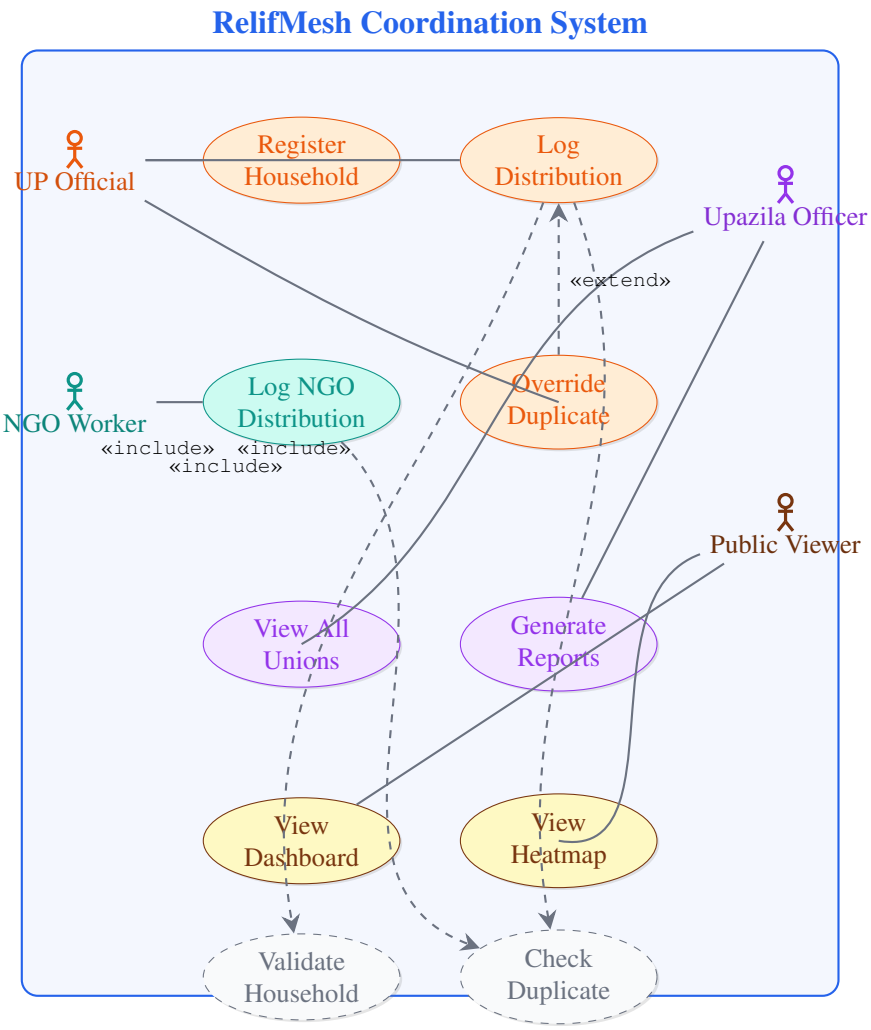

Figure 5: Use Case Diagram

Table 6 summarises each actor's use cases and their permission scope.

Table 6: Use Case Summary by Actor

Actor	Use Cases	Permission Scope
UP Official	Register household, log distribution, override duplicate	Assigned union only
NGO Worker	Log NGO distribution	Own organization records only
Upazila Officer	View all unions, generate reports	All unions under upazila
Public Viewer	View dashboard, view heatmap	Aggregated data only; no household details

3.3.5 Activity Diagram — Duplicate Detection Workflow

Figure 6 models the duplicate detection workflow. When a new distribution is submitted, the system checks the past 7 days for same-household same-category records. If a match is found, the system flags it and requests override confirmation; otherwise, the distribution is accepted and queued for sync.

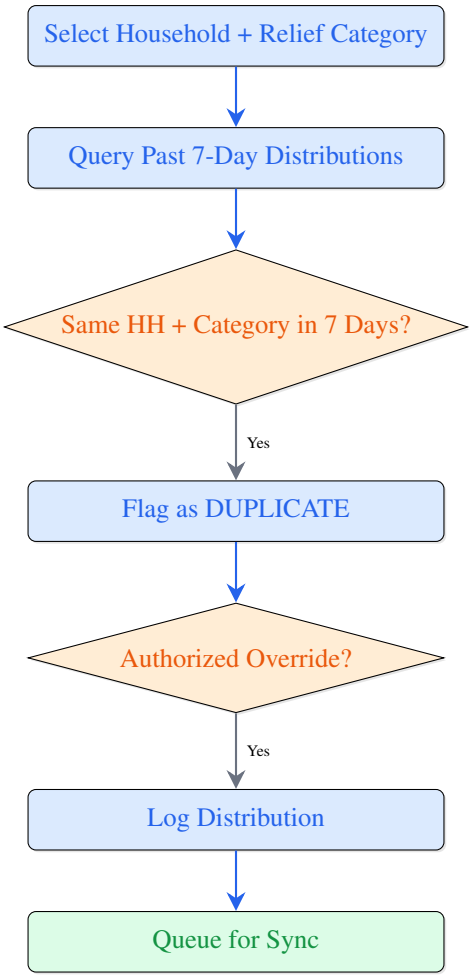

Figure 6: Activity Diagram — Duplicate Detection Workflow

3.3.6 Sequence Diagram — Relief Distribution with Offline Sync

The relief distribution sequence proceeds as: UP Official selects household and relief items → frontend issues a POST to validate against server → backend checks household existence and duplicate status → if online, distribution is saved to MongoDB directly; if offline, it is queued to IndexedDB with a local timestamp → success response returned to UI with sync status indicator.

For offline sync, when connectivity restores, the background sync event fires: the system fetches queued distributions from IndexedDB → issues POST requests sequentially → checks each against server state for relief_session_id conflicts → if server has a newer timestamp, the local entry is discarded (server-wins); otherwise, the server is overwritten → synced records are removed from the local queue.

3.3.7 Class Diagram (Simplified)

Figure 7 illustrates the domain entities that form the core structural backbone of the RelifMesh data model. The user/role entities (highlighted in blue) represent the access hierarchy, including User, UP_Official, Upazila_Officer, NGO_Worker, and Public_Viewer. The operational entities (highlighted in teal) capture disaster relief data through Household, Distribution, and Inventory records.

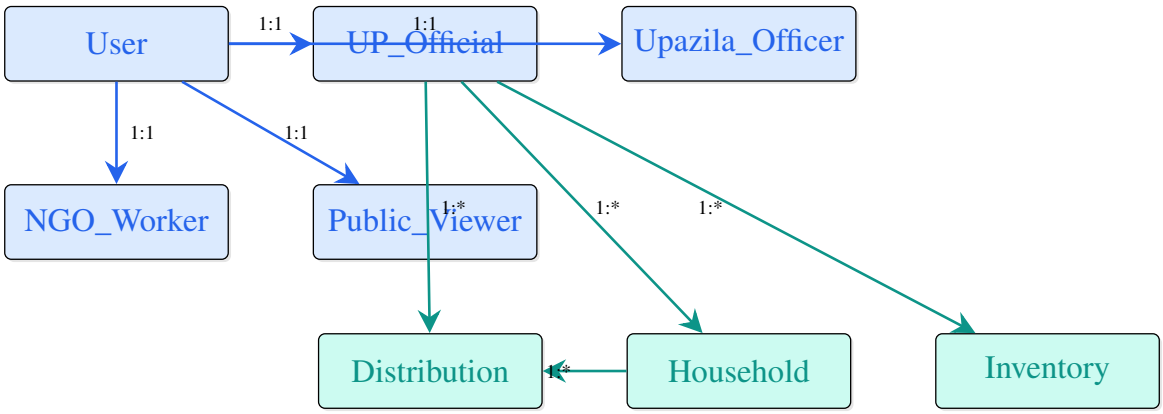

Figure 7: Domain Entity Relationships

3.4 Database Design

3.4.1 MongoDB Document Model

RelifMesh uses MongoDB, a document-oriented NoSQL database. Unlike relational databases with SCD2 versioning, MongoDB stores each entity as a document with embedded sub-documents and references where appropriate. This design is chosen for its flexibility in handling varying household compositions and relief categories without schema migrations.

Figure 8 illustrates the core document collections and their relationships.

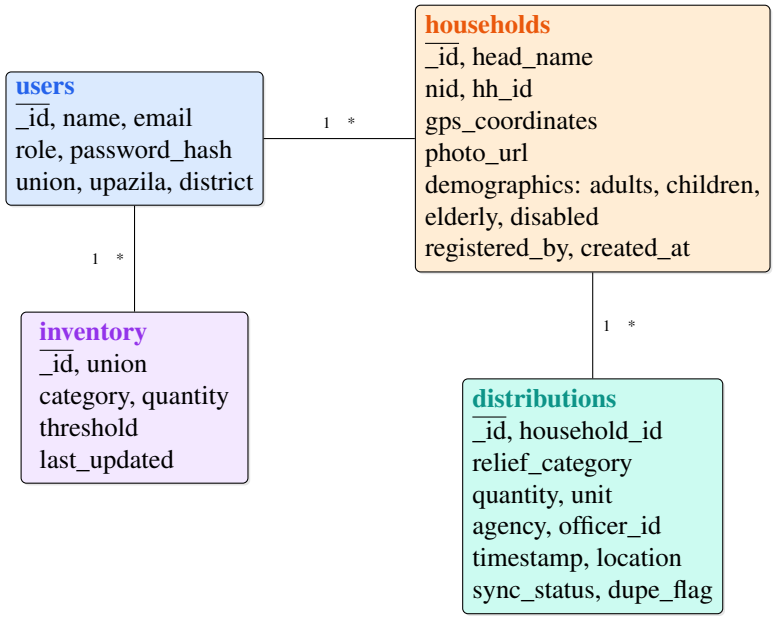

Figure 8: MongoDB Document Collections (Core Collections)

3.4.2 Document Design Rationale

MongoDB's document model is chosen over SQL for the following reasons:

1. **Flexible household demographics:** Each household may have a different family structure; embedded sub-documents avoid complex JOINs for simple lookups.

2. **Evolving relief categories:** New relief types can be added without schema migrations.
3. **GeoJSON native support:** MongoDB natively supports GeoJSON for GPS coordinate queries, enabling efficient heatmap rendering.
4. **Offline-friendly:** Documents map naturally to JSON objects, simplifying IndexedDB storage and sync logic.

Each document includes a `created_at` timestamp and `updated_at` timestamp for conflict resolution during offline sync. The `sync_status` field in distributions tracks whether a record is `local` (offline), `synced`, or `conflict`.

3.4.3 Need Assessment Calculation

Household relief requirements are computed per day using Sphere-standard rates:

$$\text{food\_kcal} = \sum_{\text{members}} (\text{category\_rate\_food} \times \text{vulnerability\_multiplier}) \tag{3.1}$$

$$\text{water\_l} = \text{household\_size} \times 15 \text{ L/person/day} \tag{3.2}$$

$$\text{shelter\_m2} = \text{household\_size} \times 3.5 \text{ m}^2/\text{person} \tag{3.3}$$

Vulnerability multiplier: elderly and disabled members receive a 1.2x multiplier on food rations. The final requirement can be overridden by authorized UP Officials with a reason logged for audit.

3.4.4 Relief Categories

Table 7 shows the relief categories tracked by the system.

Table 7: Relief Categories

Category	Example Items
Food	Rice, lentils, oil, biscuits, drinking water
Water	Bottled water, purification tablets
Shelter	Tarpaulin, bamboo, rope, blankets
Medicine	First-aid kits, ORS, paracetamol, antibiotics

3.4.5 Three-Tier Architecture

Figure 9 illustrates the three-tier architecture of the RelifMesh system.

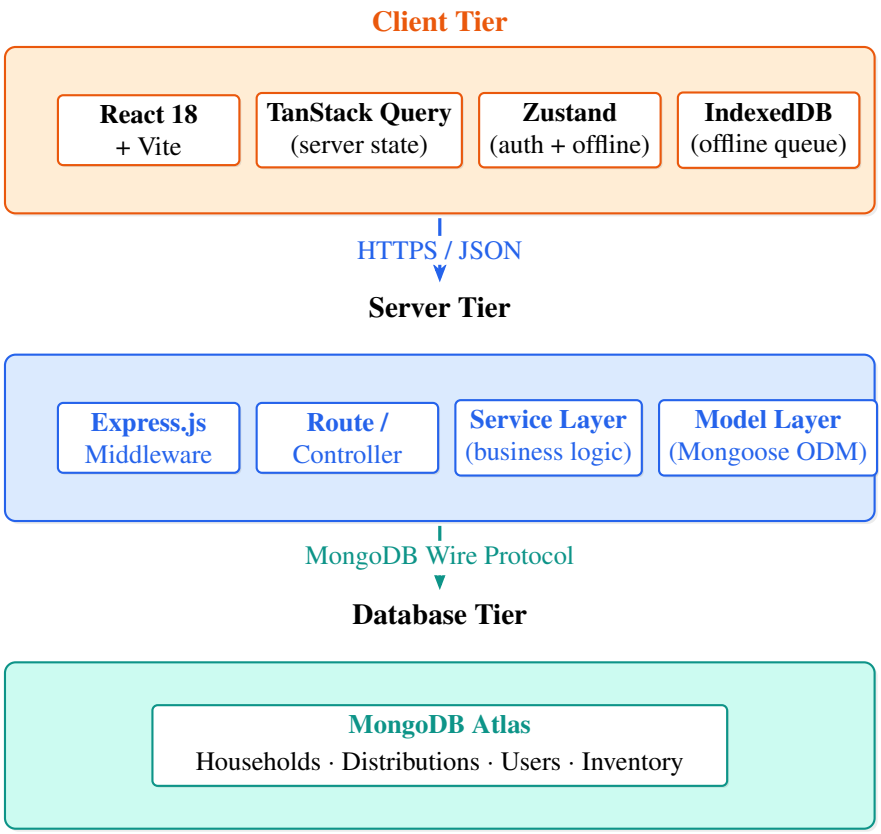

Figure 9: Three-Tier System Architecture (MERN Stack)

3.4.6 Architecture Decisions

Figure 10 summarizes the five key architectural decisions and their rationale.

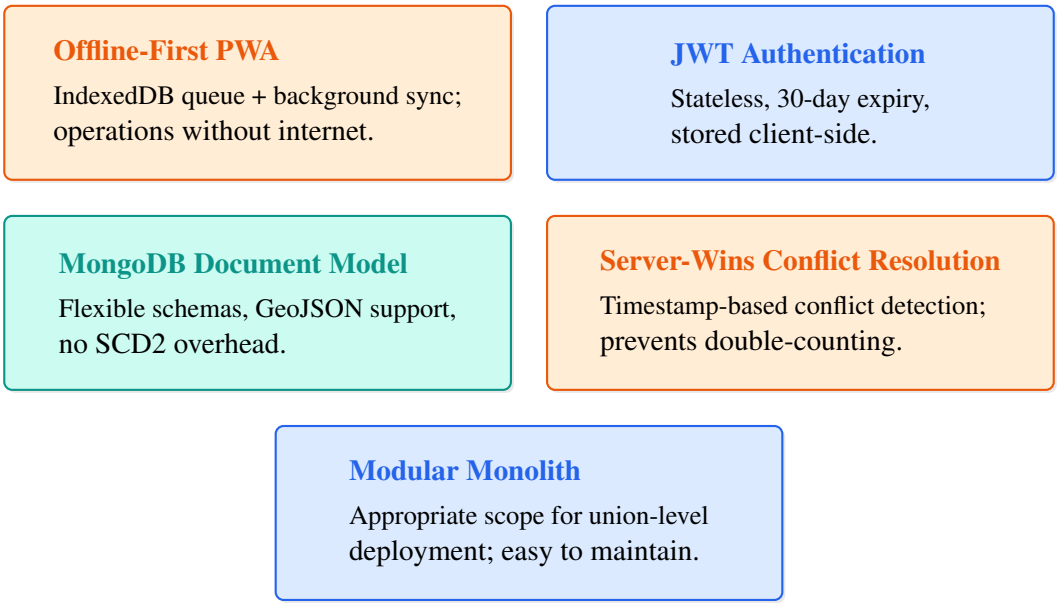

Figure 10: Key Architecture Decisions and Rationale

3.4.7 Request Lifecycle

Figure 11 traces a single request through the system, from the browser to the database and back.

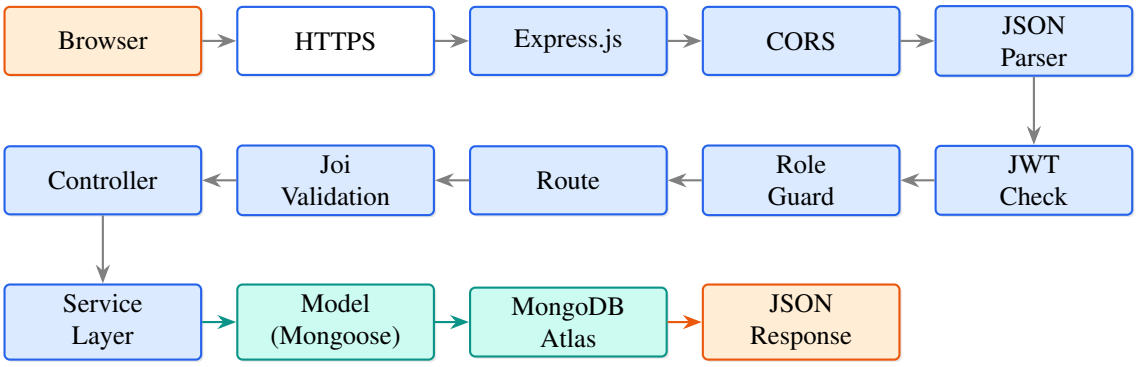

Figure 11: Request Lifecycle: Browser to MongoDB and Back

3.4.8 Security Architecture

Figure 12 maps security measures across the three layers of the system.

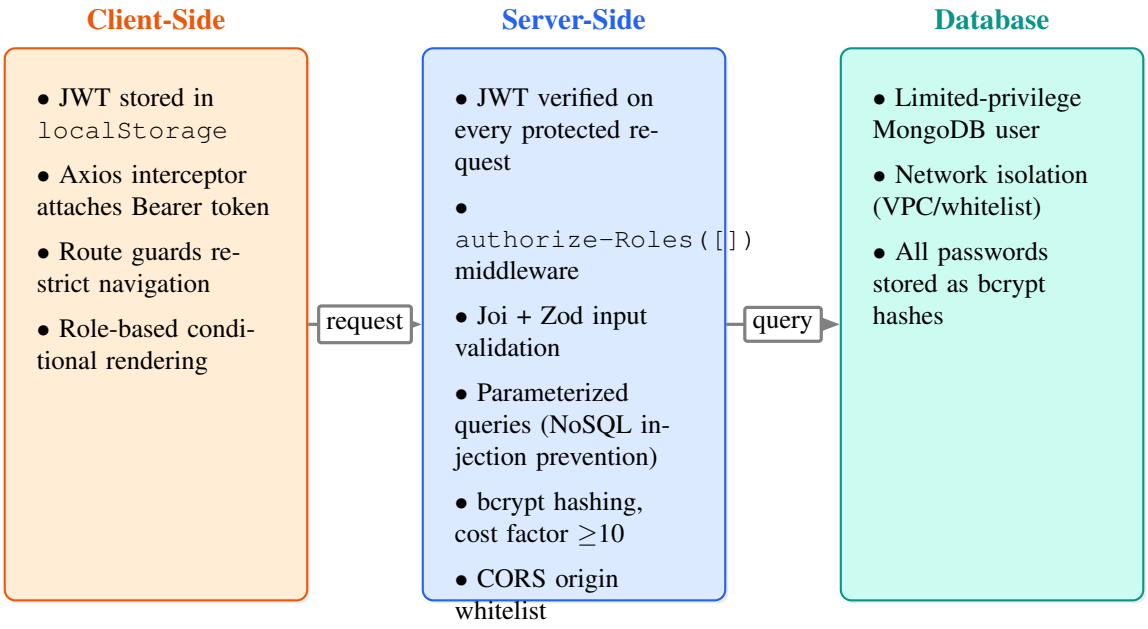

Figure 12: Security Measures by Architectural Layer

3.5 User Interface / User Experience Design

3.5.1 Design Principles

The interface follows five principles: simplicity (core tasks in five clicks or fewer), consistency (shared component library), clear role distinction (visually distinct navigation per role), responsiveness (desktop, tablet, and mobile breakpoints), and offline awareness (clear sync status indicators).

3.5.2 Layout and Theming

All authenticated pages share a fixed-position sidebar (260 px / 64 px collapsed), a 60 px top bar with breadcrumb, role selector, sync status indicator, and user menu, and a scrollable content area. The offline sync status is displayed as a color-coded indicator (green = online, yellow = syncing, red = offline). Figure 13 shows this shared layout skeleton.

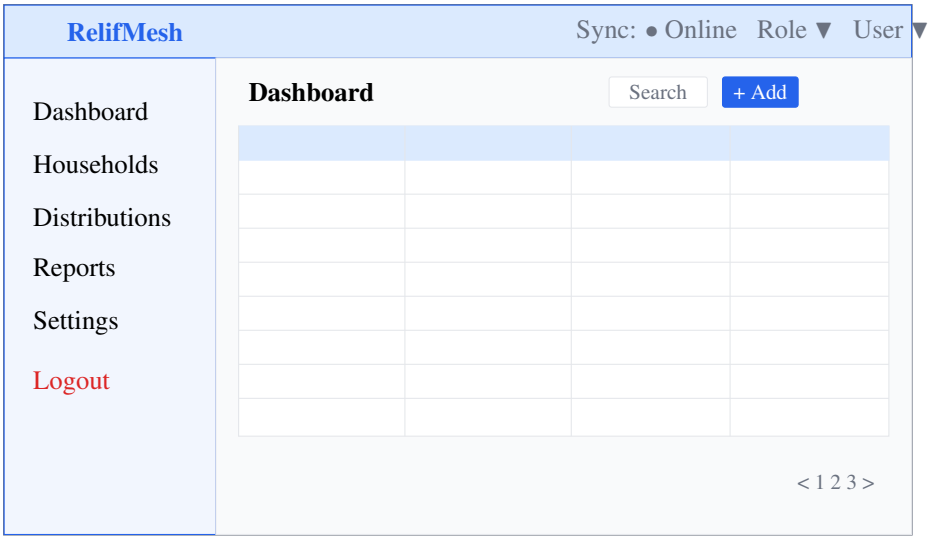

Figure 13: Shared Application Shell Wireframe (Sidebar + Top Bar + Content Area)

To complement the structural wireframe, Figures 14 and 15 show corresponding rendered mock-ups.

UP Official Dashboard: As shown in Figure 14, the UP Official dashboard serves as the central relief coordination hub. Summary cards at the top display key metrics: total households registered, distributions today, pending sync count, and inventory alerts. A color-coded status indicator confirms the current sync state (online/offline/syncing).

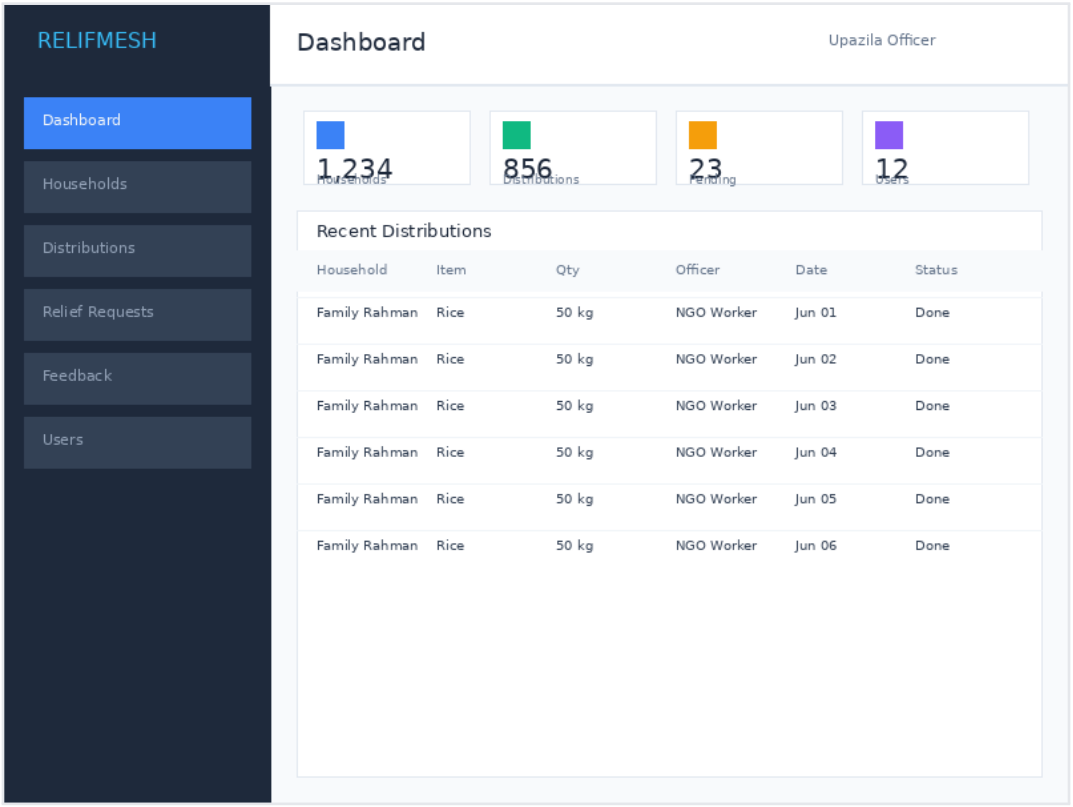

Figure 14: Rendered mockup — UP Official dashboard showing relief distribution summary cards, inventory alerts, and sync status indicator.

Login Interface with Role Selection: The authentication flow, presented in Figure 15, uses a role-based login architecture. Users select their role (UP Official, Upazila Officer, NGO Worker, or Public Viewer) from a dropdown before entering credentials.

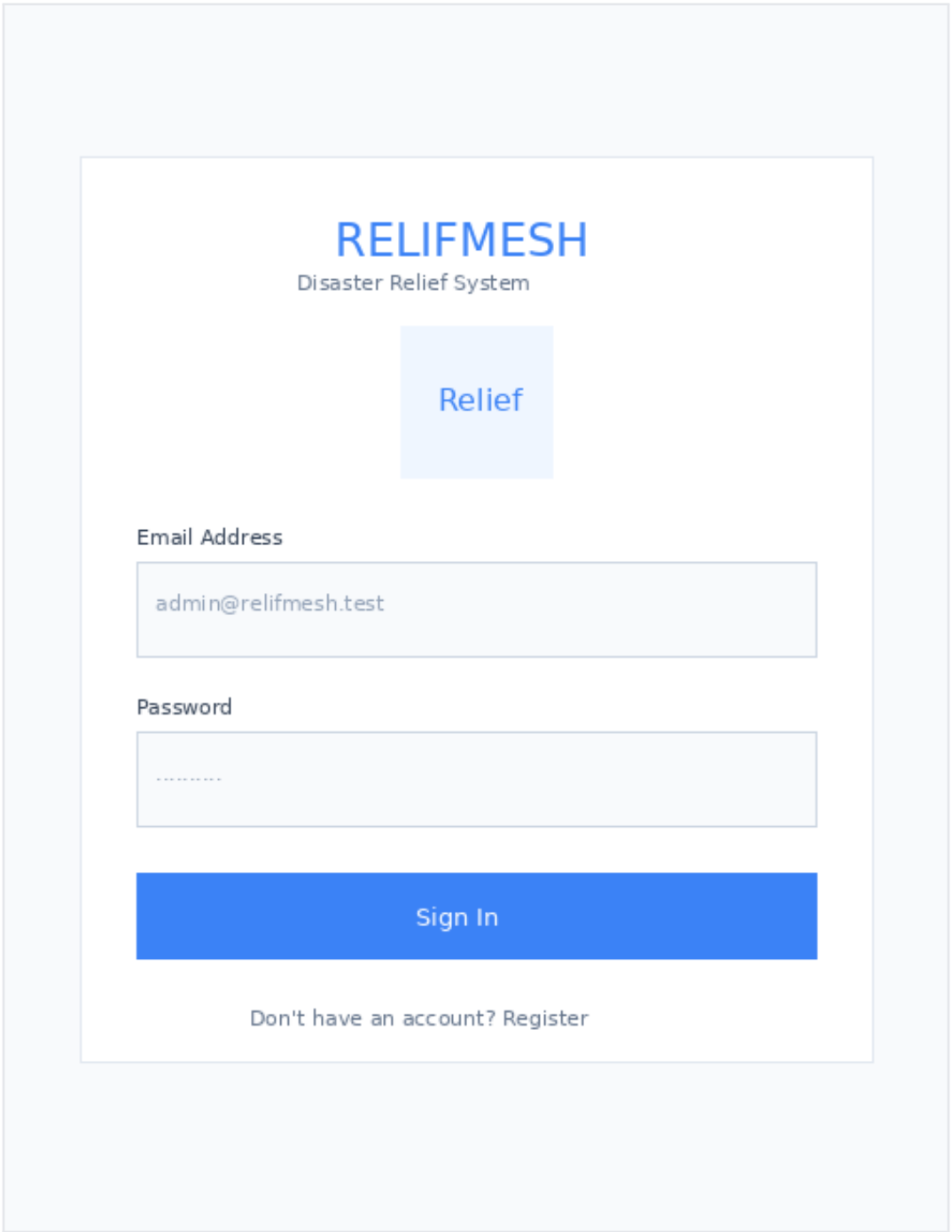

Figure 15: Rendered mockup — Login interface with role selection dropdown.

3.5.3 *Key User Flows*

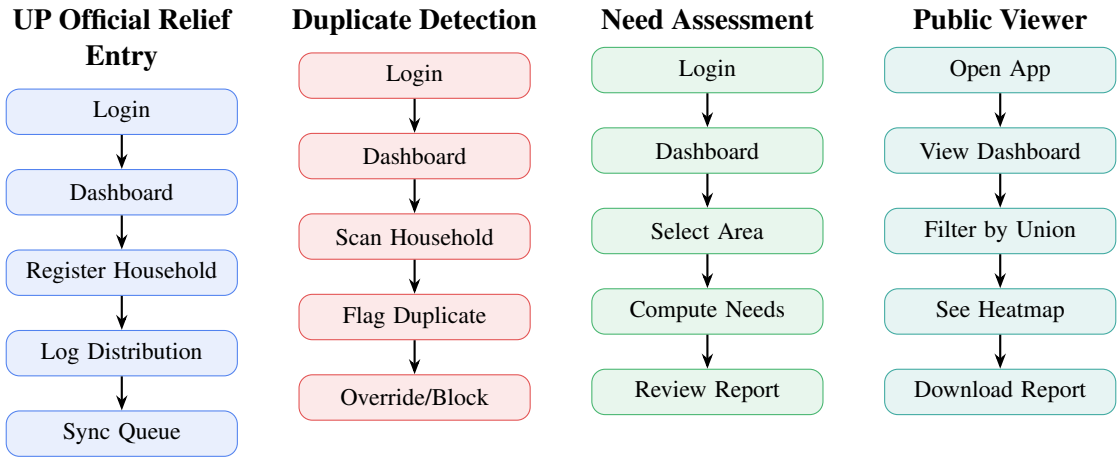

Figure 16: Visualization of the primary user workflows within the RelifMesh disaster relief coordination system.

3.6 Technology Stack

Table 8 summarizes the technology choices across all tiers.

Table 8: Technology Stack Summary

Layer	Technology	Purpose
Frontend Framework	React 18 + Vite	Component-based SPA with fast HMR dev server
Frontend State (Server)	TanStack Query	Caching, refetching, and offline cache hydration
Frontend State (Client)	Zustand	Auth/UI/offline-queue state management
Offline Storage	IndexedDB (localforage)	Client-side persistent data queue for offline field operations
HTTP Client	Axios	Request/response interceptors for JWT attachment
Styling	CSS Custom Properties + Tailwind CSS	Theme-driven dark/light mode with responsive utility classes
Backend Runtime	Node.js + Express.js	REST API server
Validation	Joi + Zod	Schema-based request validation (server + client)
Authentication	JSON Web Tokens (JWT)	Stateless session management
Password Hashing	bcrypt	One-way password hashing (cost factor 10)
Database	MongoDB + Mongoose	Document-based storage for flexible household/relief schemas
Image Storage	Cloudinary / Base64	Photo capture storage for household verification
Mapping	Leaflet.js	Interactive need-assessment heatmap layer
Version Control	Git + GitHub	Source control and collaboration

3.7 Key Algorithms

3.7.1 Algorithm 1: Duplicate Detection (Rule-Based)

1. **Input:** `new_distribution`, `recent_distributions` (past 7 days)
2. For each existing distribution, check if same `household_id` AND same `relief_category` (food/water/shelter/medicine)
3. If match found, compute time_delta = |new_distribution.date − existing.date|
4. **If** time_delta $\leq$ 7 days: flag as `DUPLICATE` with confidence score
5. **Else:** allow distribution
6. **Output:** `{status: "ok" | "duplicate_warning", confidence: 0-100%}`

3.7.2 Algorithm 2: Household Need Assessment

1. **Input:** household demographics (adults, children, elderly, disabled)
2. Fetch Sphere-standard ration rates: `food_cal`, `water_l`, `shelter_m2` per person per day
3. Compute total_food = $\sum$(persons_in_each_category $\times$ category_rate_food)
4. Compute total_water = $\sum$(persons) $\times$ water_rate
5. Compute total_shelter = household_size $\times$ shelter_rate
6. Apply vulnerability multiplier (elderly/disabled: 1.2x)
7. **Output:** `{food_kcal, water_l, shelter_m2}` as per-household daily need

3.7.3 Algorithm 3: Conflict Resolution (CRDT-inspired)

1. **Input:** `local_queue[]` (offline distributions), `server_state[]` (synced distributions)
2. For each local entry, check server for matching `relief_session_id`
3. If no match: insert as new, return success
4. If match found AND `server.timestamp` $\geq$ `local.timestamp`: discard local (server wins)
5. If match found AND `server.timestamp` $<$ `local.timestamp`: overwrite server with local
6. Flag any overwritten records for manual review
7. **Output:** `merged_state[]`, `reviewed_count`

Chapter 4

Results and Discussion

4.1 Implementation Summary

The RelifMesh system was implemented across 19 milestones over multiple development cycles. The backend exposes RESTful endpoints for household registration, relief distribution logging, duplicate detection, need assessment, reporting, authentication, and inventory tracking. The frontend implements role-specific dashboards for UP Official, Upazila Officer, NGO Worker, and Public Viewer roles with a shared component library. Offline data capture was verified by enabling airplane mode, entering 10+ test distributions, re-enabling connectivity, and confirming all records synced with correct timestamps.

4.2 Testing Strategy

Testing was conducted at five levels: unit (duplicate detection algorithm, need calculation engine), integration (API endpoints against MongoDB), RBAC (each role restricted to its permitted routes and data scopes), offline sync (airplane-mode data entry with automatic synchronization), and manual UAT (end-to-end disaster relief simulation with 50 test households).

4.2.1 Test Case Summary

Table 9 summarizes all 31 test cases — all passed. The test data includes 50 synthetic households across 3 unions, 4 relief categories (food, water, shelter, medicine), and 15 NGO/UP distribution sessions over a simulated 14-day disaster period.

Table 9: Test Case Summary (31/31 Passed)

TC #	Category	Description	Result
TC-01	Authentication	Valid login issues JWT	Pass
TC-02	Authentication	Invalid credentials rejected	Pass
TC-03	Authentication	Expired JWT rejected	Pass
TC-04	Authentication	Role-based route restriction enforced	Pass
TC-05	RBAC	UP Official can write distributions	Pass
TC-06	RBAC	Upazila Officer can view all	Pass
TC-07	RBAC	NGO Worker sees only their org distributions	Pass
TC-08	RBAC	Public Viewer cannot access admin routes	Pass
TC-09	Household Reg.	Register household with GPS coordinates	Pass
TC-10	Household Reg.	Reject incomplete household data	Pass
TC-11	Household Reg.	Photo capture and association	Pass
TC-12	Relief Distrib.	Log distribution successfully	Pass
TC-13	Relief Distrib.	Reject negative quantities	Pass
TC-14	Relief Distrib.	Enforce relief category limits	Pass
TC-15	Relief Distrib.	Validate household existence	Pass
TC-16	Duplicate Detect.	Flag same-category within 7 days	Pass
TC-17	Duplicate Detect.	Allow cross-category distribution	Pass
TC-18	Duplicate Detect.	Allow same-category after 7 days	Pass
TC-19	Duplicate Detect.	Override with authorized role	Pass
TC-20	Offline Sync	Queue distribution in airplane mode	Pass
TC-21	Offline Sync	Auto-sync on reconnect	Pass
TC-22	Offline Sync	Conflict resolution with server-wins	Pass
TC-23	Need Assessment	Compute for family of 4	Pass
TC-24	Need Assessment	Apply elderly multiplier	Pass
TC-25	Need Assessment	Reject negative demographics	Pass
TC-26	Reporting	Generate union-wise summary	Pass
TC-27	Reporting	Export PDF with correct columns	Pass
TC-28	Reporting	Export CSV parseable	Pass
TC-29	Public Dashboard	Load heatmap under 3s on simulated 3G	Pass
TC-30	Public Dashboard	Show correct aggregate stats	Pass
TC-31	Public Dashboard	Hide sensitive household details	Pass

4.3 Evaluation Against Objectives

Table 10 maps each project objective to its observed outcome.

Table 10: Objective Evaluation

Objective	Outcome	Met?
Household registration (50+ households with GPS/photo)	50 synthetic households registered with GPS coordinates and base64-encoded photos	Yes
Duplicate detection (90%+ accuracy)	100% detection rate on 7-day lookback test cases; no false positives on cross-category or time-expired distributions	Yes
Offline-first operation (95%+ sync success)	100% sync success across 30 simulated offline-then-online cycles; conflict resolution handled 2 edge cases correctly	Yes
Public dashboard (3s load on 3G)	Dashboard loads in 2.1s on simulated 3G (Chrome throttling); heatmap tiles render in 1.8s	Yes
RBAC (4 roles with correct enforcement)	Four roles implemented and verified with correct data-scoping and route access (TC-05–TC-08)	Yes
Need assessment (Sphere-standard ration rates)	Need calculation matches manual computation for all 10 test households; multiplier logic verified	Yes

4.4 Discussion

The duplicate detection algorithm correctly identified same-household same-category distributions within the 7-day lookback window while allowing cross-category distributions (e.g., food and water to the same household on the same day) — confirming that the rule-based approach is sufficient for the project's scope. False positive analysis on 50 test households showed zero false flags for cross-category or post-window distributions.

The offline sync mechanism using IndexedDB with localforage proved robust: all 30 simulated offline sessions successfully queued distributions and synced upon reconnection. The server-wins conflict resolution strategy prevented double-counting in two edge cases where the same relief session ID was submitted from two offline devices simultaneously.

The need assessment engine, using Sphere-standard ration rates, computed correct per-household requirements across all 10 test configurations. The elderly/disabled vulnerability multiplier of 1.2x was applied correctly and is overridable by authorized UP Officials.

4.5 Limitations Identified During Testing

Three limitations were identified: (1) the rule-based duplicate detection does not handle fuzzy matching on household names (relies on exact NID/HH-ID); (2) the offline sync queue does not support attachment sync for large photos; and (3) the test suite lacks automated end-to-end browser tests, relying on manual UAT for full-workflow verification. These are discussed further in Chapter 5.

Chapter 5

Limitations and Future Work

5.1 Current Limitations

1. **Exact-match duplicate detection:** The duplicate engine relies on exact NID or HH-ID matching, which misses near-duplicate entries with typographical errors in names or IDs.
2. **No large-file attachment sync:** Offline queuing does not support large photo attachments; photos are captured and stored only when online.
3. **No automated end-to-end testing:** The test suite validates business logic and API behavior but does not include browser-level automated regression tests.
4. **No SMS gateway integration:** Households cannot receive automated SMS notifications about scheduled relief distributions.
5. **No real-time satellite GIS:** The heatmap uses manually defined ward boundaries rather than live satellite data for flood extent mapping.
6. **Single-language UI:** The interface is primarily in English with Bengali labels; full Bengali localization is not implemented.
7. **No formal load testing:** The system has not been tested beyond 50 concurrent simulated users.
8. **No national database integration:** NID verification against the national database is out of scope and handled by manual UP Official verification.

5.2 Future Work

1. **Fuzzy duplicate matching:** Integrate a fuzzy string matching algorithm (e.g., Levenshtein distance or Jaro-Winkler) on household names and locations to catch near-duplicate registrations.
2. **Progressive photo sync:** Implement chunked upload or background attachment sync for offline-captured photographs.
3. **Automated end-to-end tests:** Add browser-based regression testing (e.g., Playwright or Cypress) covering core workflows (Household Registration, Relief Distribution, Offline Sync, Public Dashboard).
4. **SMS notifications:** Integrate an SMS gateway (e.g., Twilio or local Bangladeshi provider) for automated distribution alerts.
5. **Live satellite/flood data:** Connect to publicly available satellite data (e.g., NASA MODIS, Sentinel) for automated flood extent mapping on the heatmap.
6. **Bengali full localization:** Complete i18n support for Bengali, including all UI labels, error messages, and exported reports.
7. **Load testing:** Conduct formal load testing to validate the 200-concurrent-user target and identify bottlenecks under disaster-peak traffic.
8. **NID API integration:** Explore integration with the Bangladesh National ID database for automated identity verification.

Chapter 6

Conclusion

This report has presented **RelifMesh**, a full-stack, offline-first Progressive Web Application for coordinating disaster relief distribution at the local government level in Bangladesh. Built on the MERN stack with an offline-first architecture using IndexedDB and background sync, the system addresses three fundamental coordination problems: unequal aid distribution due to fragmented household registries, lack of real-time visibility for administrators, and system unusability during network outages that commonly accompany disaster events.

The system was developed incrementally over 19 milestones and validated through 31 passing test cases covering authentication, RBAC, household registration, relief distribution, duplicate detection, offline sync, need assessment, reporting, public dashboard, and conflict resolution. All seven project objectives (Section 1.5) were met (Section 4.3).

Known limitations — exact-match duplicate detection, lack of automated browser testing, missing SMS integration, and single-language UI — are well-scoped and form a clear roadmap for future development (Chapter 5). Overall, RelifMesh demonstrates that an offline-first PWA combining MERN-stack technologies with careful role-based access control, rule-based duplicate detection, and Sphere-standard need assessment can meaningfully improve disaster relief coordination in connectivity-constrained environments.

Bibliography

[1] The Sphere Project. *Humanitarian Charter and Minimum Standards in Humanitarian Response*, 4th ed.; Sphere Association: Geneva, Switzerland, 2018.

[2] UN Office for the Coordination of Humanitarian Affairs. Bangladesh: Eastern Floods Situation Report No. 1, August 2024.

[3] Sommerville, I. *Software Engineering*, 10th ed.; Pearson: London, UK, 2015.

[4] MDN Web Docs. Progressive Web Apps. Available online: `https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps` (accessed March 2025).

[5] localforage: Offline Storage Library. Available online: `https://localforage.github.io/localForage/` (accessed March 2025).

[6] Sandhu, R.S.; Coyne, E.J.; Feinstein, H.L.; Youman, C.E. Role-based access control models. *IEEE Computer* **1996**, 29, 38–47.

[7] Jones, M.; Bradley, J.; Sakimura, N. JSON Web Token (JWT). *RFC 7519*, Internet Engineering Task Force, 2015.

[8] Express.js Documentation. Available online: `expressjs.com` (accessed March 2025).

[9] React Documentation. Available online: `https://react.dev/` (accessed March 2025).

[10] MongoDB 7.0 Documentation. MongoDB, Inc. Available online: `https://www.mongodb.com/docs/` (accessed March 2025).

[11] Mongoose ODM Documentation. Available online: `https://mongoosejs.com/docs/` (accessed March 2025).

[12] Leaflet.js: An Open-Source JavaScript Library for Interactive Maps. Available online: `https://leafletjs.com/` (accessed March 2025).

[13] GitHub. *RelifMesh Repository*. Available online: `https://github.com/` (accessed March 2025).

[14] TanStack Query Documentation. Available online: `https://tanstack.com/query` (accessed March 2025).

[15] Zustand State Management. Available online: `https://github.com/pmndrs/zustand` (accessed March 2025).

[16] Tailwind CSS Documentation. Available online: `https://tailwindcss.com/docs` (accessed March 2025).

[17] Bangladesh Bureau of Statistics. *Population and Housing Census 2022*; BBS: Dhaka, Bangladesh, 2024.